

Basic Complete Guide to Linked Lists in Data Structures

1. What is a Linked List?

A **linked list** is a linear data structure made of **nodes**, where each node stores:

1. **Data** (the value)
2. A **pointer** (address) to the next node (and sometimes the previous node)

Unlike arrays, linked lists do **not** require contiguous memory. Nodes can live anywhere in memory, and pointers connect them together.

Why it exists

Arrays are great for fast indexing, but expensive for insertions/deletions in the middle because elements must be shifted. Linked lists make insert/delete easier (when you already have the correct pointer) by changing links instead of shifting elements.

2. Linked List Node Structure (C++)

Singly Linked List Node

```
struct Node {  
    int data;  
    Node* next;  
  
    Node(int val) : data(val), next(nullptr) {}  
};
```

- **data**: stores the integer (could be any type)
- **next**: points to the next Node

- `nullptr` means “no next node” (end of list)
-

3. Visual Model

Example list:

`10 -> 20 -> 30 -> nullptr`

- Head points to the first node
 - Each node points to the next
 - Last node points to `nullptr`
-

4. Types of Linked Lists

A) Singly Linked List

Each node points only forward (`next`).

B) Doubly Linked List

Each node points forward and backward (`next`, `prev`).

C) Circular Linked List

Last node points back to the head instead of `nullptr`.

Most DS courses start with **singly linked lists**, so that's what this guide focuses on.

5. Core Operations and Big-O

Let `n` be the number of nodes.

Operation	Time Complexity	Why
Access by index i	$O(n)$	must traverse from head
Search for value	$O(n)$	worst case scan entire list
Insert at head	$O(1)$	constant pointer updates
Delete at head	$O(1)$	move head pointer
Insert at tail	$O(n)$ ($O(1)$ if tail pointer)	need to find last node
Delete by value	$O(n)$	find node first
Insert/delete after a known node	$O(1)$	pointer updates only

6. Full Singly Linked List Implementation (C++)

This is a clean “class-based” version with common methods:

- insert at front
- insert at back
- search
- delete value
- print
- destructor to free memory

```
#include <iostream>
using namespace std;
```

```
class LinkedList {
```

```

private:
    struct Node {
        int data;
        Node* next;
        Node(int val) : data(val), next(nullptr) {}
    };

    Node* head;

public:
    LinkedList() : head(nullptr) {}

    // Destructor: free all nodes to avoid memory leaks
    ~LinkedList() {
        Node* curr = head;
        while (curr != nullptr) {
            Node* nextNode = curr->next;
            delete curr;
            curr = nextNode;
        }
    }

    // Print list: O(n)
    void print() const {
        Node* curr = head;
        while (curr != nullptr) {
            cout << curr->data;
            if (curr->next != nullptr) cout << " -> ";
            curr = curr->next;
        }
        cout << " -> nullptr\n";
    }

    // Insert at front: O(1)
    void pushFront(int val) {
        Node* newNode = new Node(val);
        newNode->next = head;
        head = newNode;
    }

```

```

}

// Insert at back: O(n)
void pushBack(int val) {
    Node* newNode = new Node(val);

    if (head == nullptr) {
        head = newNode;
        return;
    }

    Node* curr = head;
    while (curr->next != nullptr) {
        curr = curr->next;
    }
    curr->next = newNode;
}

// Search: returns true if found, O(n)
bool contains(int target) const {
    Node* curr = head;
    while (curr != nullptr) {
        if (curr->data == target) return true;
        curr = curr->next;
    }
    return false;
}

// Delete first occurrence of value: O(n)
bool remove(int target) {
    if (head == nullptr) return false;

    // If head needs to be removed
    if (head->data == target) {
        Node* temp = head;
        head = head->next;
        delete temp;
        return true;
    }

```

```

    }

    // Find node before the one to delete
    Node* prev = head;
    Node* curr = head->next;

    while (curr != nullptr) {
        if (curr->data == target) {
            prev->next = curr->next;
            delete curr;
            return true;
        }
        prev = curr;
        curr = curr->next;
    }

    return false; // not found
}

};

int main() {
    LinkedList list;

    list.pushBack(10);
    list.pushBack(20);
    list.pushBack(30);
    list.print(); // 10 -> 20 -> 30 -> nullptr

    list.pushFront(5);
    list.print(); // 5 -> 10 -> 20 -> 30 -> nullptr

    cout << "Contains 20? " << (list.contains(20) ? "Yes" : "No") <<
endl;
    cout << "Contains 99? " << (list.contains(99) ? "Yes" : "No") <<
endl;

    list.remove(10);
    list.print(); // 5 -> 20 -> 30 -> nullptr

```

```
    return 0;  
}
```

7. Code Walkthrough (How the important parts work)

A) Insert at Front (pushFront)

Steps:

1. Create new node
2. Point it to current head
3. Update head to new node

That's why it's $O(1)$.

B) Insert at Back (pushBack)

Steps:

1. If list empty, head becomes new node
2. Otherwise traverse until last node
3. Attach new node to the end

Traversal makes it $O(n)$.

C) Remove a Value (remove)

There are two common cases:

- removing the head (special case)
 - removing somewhere in the middle/end (need `prev` pointer)
-

8. Common Interview/Class Questions (With Examples)

A) Find Length of Linked List (Iterative)

```
int length(Node* head) {
    int count = 0;
    Node* curr = head;
    while (curr != nullptr) {
        count++;
        curr = curr->next;
    }
    return count;
}
```

Time: $O(n)$

B) Reverse a Linked List (Iterative, Classic DS)

Idea: flip pointers one by one.

```
Node* reverse(Node* head) {
    Node* prev = nullptr;
    Node* curr = head;

    while (curr != nullptr) {
        Node* nextNode = curr->next; // save next
        curr->next = prev;           // reverse link
        prev = curr;                 // move prev
        curr = nextNode;             // move curr
    }
    return prev; // new head
}
```

Time: $O(n)$

Space: $O(1)$

C) Detect a Cycle (Floyd's Tortoise and Hare)

If there is a loop (cycle), fast and slow pointers will meet.

```
bool hasCycle(Node* head) {
    Node* slow = head;
    Node* fast = head;

    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next;          // 1 step
        fast = fast->next->next;     // 2 steps
        if (slow == fast) return true;
    }
    return false;
}
```

Time: O(n)

Space: O(1)

9. Common Mistakes Students Make

- Forgetting to update `head` when inserting/deleting at front
 - Losing the list by overwriting pointers without saving `next`
 - Memory leaks: creating nodes but never deleting them
 - Not handling empty list cases (`head == nullptr`)
 - Off-by-one traversal (stopping at wrong node)
-

10. Linked List vs Array (When to Use Which)

Linked List is better when:

- Many insertions/deletions near head or known node
- You don't know size ahead of time
- You want $O(1)$ insert/delete at front

Array is better when:

- You need fast indexing `arr[i]` ($O(1)$)
 - Cache performance matters (arrays are contiguous)
 - You don't insert/delete often in the middle
-